



Discrete Optimization

Variable neighbourhood search for bandwidth reduction

Nenad Mladenovic^{a,*}, Dragan Urošević^b, Dionisio Pérez-Brito^c, Carlos G. García-González^d^a School of Mathematics, Brunel University – West London, UK^b Mathematical Institute SANU Belgrade, Serbia^c Dpto. de Estadística, Investigación Operativa y Computación, Universidad de La Laguna, Spain^d Dpto. de Economía de las Instituciones, Estadística Económica y Econometría, Universidad de La Laguna, Spain

ARTICLE INFO

Article history:

Received 23 March 2008

Accepted 9 December 2008

Available online 24 December 2008

Keywords:

Combinatorial optimization

Matrix bandwidth minimization

Metaheuristics

Variable neighbourhood search

ABSTRACT

The problem of reducing the bandwidth of a matrix consists of finding a permutation of rows and columns of a given matrix which keeps the non-zero elements in a band as close as possible to the main diagonal. This NP-complete problem can also be formulated as a vertex labelling problem on a graph, where each edge represents a non-zero element of the matrix. We propose a variable neighbourhood search based heuristic for reducing the bandwidth of a matrix which successfully combines several recent ideas from the literature. Empirical results for an often used collection of 113 benchmark instances indicate that the proposed heuristic compares favourably to all previous methods. Moreover, with our approach, we improve best solutions in 50% of instances of large benchmark tests.

Crown Copyright © 2008 Published by Elsevier B.V. All rights reserved.

1. Introduction

Let $G(V, E)$ be a graph with node set V and edge set E and let $f: V \rightarrow \{1, \dots, n\}$ be a labelling (or colouring) of nodes, where $n = |V|$. Let us define the *bandwidth of a graph G under f* as

$$B(f) = \max\{|f(u) - f(v)| \mid \forall (u, v) \in E\}.$$

Moreover, if we define the *bandwidth of vertex v under f* as

$$B(f, v) = \max_{u: (u, v) \in E} |f(u) - f(v)|, \quad (1)$$

the bandwidth may also be expressed as $B(f) = \max_v B(f, v)$. Then the *bandwidth minimization problem* is to find a labelling f^* , which minimizes $B(f)$, i.e.

$$B^* = B(f^*) = \min_f B(f) = \min_f \max_{(u, v) \in E} |f(u) - f(v)|$$

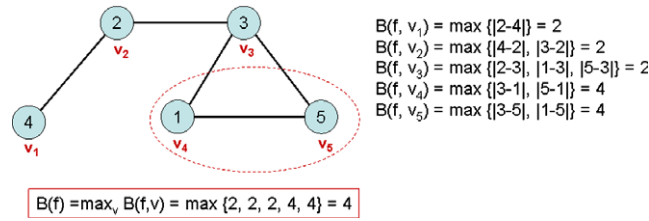
and the aim of bandwidth reduction is to find a labelling which gives an approximate solution to this problem.

If we represent G by its adjacency matrix A , the problem can be formulated as finding a permutation of the rows and the columns of A which keeps all the non-zero elements in a band as close as possible to the main diagonal. This is why this problem is known as the Matrix bandwidth minimization problem (MBMP for short), where *bandwidth* is defined as the maximum distance of a nonzero element from the main diagonal.

Example. Given a graph G with $n = 5$ and the given labelling $f: f(v_1) = 4, f(v_2) = 2, f(v_3) = 3, f(v_4) = 1, f(v_5) = 5$. As indicated in Fig. 1, the *bandwidth of G under the labelling f* is $B(f) = 4$ and the edge which produces this value is the one which is rounded. The adjacency matrix of the graph under labelling f is as follows:

* Corresponding author.

E-mail addresses: Nenad.Mladenovic@brunel.ac.uk (N. Mladenovic), draganu@turing.mi.sanu.ac.yu (D. Urošević), Dperez@ull.es (D. Pérez-Brito), Cggarcia@ull.es (C.G. García-González).

Fig. 1. Labelling f of a graph G .

$$A(f) = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

If we exchange the label of node v_1 with the label of node v_4 , the resulting graph with the new labelling f' is given in Fig. 2.

Now, it is clear that $B(f') = 2$. Thus, the bandwidth has been reduced and the corresponding adjacency matrix $A(f')$ is

$$A(f') = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

There are $n!$ possible labellings for a graph with n vertices, and thus the MBMP is a highly combinatorial problem. Papadimitriou has shown that finding the minimum bandwidth of a graph is NP-Complete [23], even for certain families of trees [7]. Since it belongs to the NP-hard class, an exact method for its solution cannot in most cases of interest address realistically sized problem instances. Many heuristics and specialized meta-heuristics have thus been proposed and moderate size problem instances may today be tackled quite efficiently. Large problem instances are, however, still difficult to address.

Bandwidth minimization has a wide range of applications in several fields, e.g., circuit design, data storage, chemical kinetics, network survivability, numerical geophysics, industrial electromagnetic and saving large hypertext media [3]. However, the main application of this problem is to solve large linear systems. Gaussian elimination can be performed in $O(n\beta^2)$ time on matrices with $n \times n$ and bandwidth β , which is faster than the straightforward $O(n^3)$ algorithm if $\beta \ll n$. Thus, in order to improve the efficiency of the polynomial algorithm (for solving linear system of equations), we need to solve an MBMP, which is an NP-hard problem. Clearly, this is only worthwhile if the heuristic method used for solving the latter problem is fast and effective.

The aim of this paper is to apply the VNS methodology for solving the MBMP. We use the fast Improved Hill Climbing (IHC) algorithm proposed in [16], as the local search ingredient. We perturb the incumbent locally optimal solution by using two types of distance functions: reduced swap, as in [19], and rotation, as in [25]. As will be shown in the computational experiments section, the proposed procedures provide high quality solutions within a short computational time.

The next section summarizes the recent MBMP literature. Section 3 recalls the main VNS methodology and describes how the MBMP based on the VNS framework can be solved. Section 4 presents and analyzes the results of our computational experiments. Section 5 concludes the paper.

2. Previous contributions

Given its importance, the bandwidth minimization problem has been the object of extensive research, which can be divided into two classes: exact and heuristic algorithms. Exact algorithms guarantee always to discover the optimal bandwidth. An algorithm which solves the MBMP in $O(N^\beta)$ steps, where β is the bandwidth searched for, is presented in [10]. In [4] two other exact algorithms are proposed. Both methods solve problems up to 100 vertices, for randomly generated graphs. Heuristic algorithms, for their part, try to find good solutions as

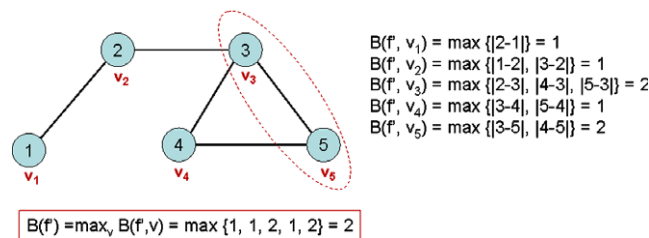
Fig. 2. Labelling f' of graph G .

Table 1
Overview of recent MBMP heuristics.

Authors	Year	Contribution	Comment
Martí et al.	2001	Tabu Search	Reduced swap neighbourhood introduced based on middle node concept
Piñana et al.	2004	GRASP with Path Relinking	Same neighbourhood as for TS used; good results reported for small size problems
Lim et al.	2006	Genetic+IHC, Node adjustment+IHC	Two methods which use efficient implementation of reduced swap local search (IHC)
Lim et al.	2007	Particle Swarm+IHC	Method which uses efficient implementation of reduced swap local search within PSO
Campos et al.	2006	Adaptive memory programming (Scatter Search, Tabu Search)	3 neighbourhoods tried out: reduced swap, 2-swap and rotation; TS is more efficient than SS
Rodriguez-Tello et al.	2008	Simulating Annealing	New representation of the solution (as $\pi = f^{-1}$); rotation-based neighbourhood introduced

fast as possible, but do not guarantee the optimality of the solution found. Table 1 summarizes the most relevant heuristics for this problem from 2001 onwards. A similar table in [19] summarizes heuristic literature for MBMP before 2001.

Most heuristics suggested in the literature include neighbourhood search. The size of a relatively large 1-interchange (or swap) neighbourhood is obviously $O(|E|)$. Martí et al. [19] proposed a way of reducing the 1-interchange neighbourhood significantly. The middle label of vertex v ($mid(v)$) is defined as an arithmetic mean of minimum and maximum distant labelled vertices adjacent to v . In this case only critical edges (u, v) with weight less than $(u, mid(v))$ are considered for swap. Based on such defined reduced swaps, a new Tabu search (TS) method is suggested. In TS terminology, a candidate list strategy was used to accelerate the selection of moves in a neighbourhood. Extensive experiments showed that the performance of their TS was better than that of previous approaches.

More recently Pinana et al. suggest in [24] a GRASP (Greedy Randomized Adaptive Search Procedure) [6] with the Path Relinking [9] method. It again uses reduced swaps. Computational results show that the method is comparable in solution quality with TS, and even better for small problems.

In the next important paper, Lim et al. [16] show how the swap neighbourhood may be further reduced, without losing its best solution. In addition, verifying if a swap has improved a solution is time-consuming since the bandwidth for swapped nodes and connected nodes must be recalculated each time a swap is executed. To address this, the amount of checking required is also reduced. Thanks to this result, an IHC algorithm is designed and two heuristics which use IHC as a subroutine are suggested:

(i) *Node adjustment* (NA) heuristic. The desired label of each vertex according to the label of its adjacent vertices is evaluated and then a way of sorting of all vertices in the graph with respect to these desired labels is found; finally all the vertices are re-labelled, following this ordering. This innovative method is repeated until no vertex changes its label and it is coupled with IHC local search. NA may be seen as a fixed neighbourhood or as an Iterated local search method, since the incumbent solution is perturbed and used as an initial solution for the local search IHC.

(ii) *Genetic Algorithm* (GA) generates solutions in the initial population with a level structure procedure and implements a classic mid-point crossover as a combination operator.

Lim et al. [17] use their previously proposed HC heuristic, but this time within a discrete particle swarm optimization (PSO) metaheuristic.

Campos et al. [2] propose two methods. One is based on Tabu Search (TS) and the other on Scatter Search (SS). These authors use three neighbourhood structures in both TS and SS: reduced swap, 2-swap and rotation, as in [25]. It is interesting to note that results obtained are of similar quality as those obtained by SA, although more neighbourhood structures were used.

Finally, Rodriguez-Tello et al. [25] present an improved Simulated Annealing (SA) heuristic. This algorithm integrates several important features such as an original internal representation of solutions, a highly discriminating evaluation function called δ and a rotation-based neighbourhood structure. Therefore, their work can be viewed as guided or dynamic local search, since the evaluating function can change during the search. The main reason for the very good results reported is probably the fact that the new presentation and the rotation move allow smaller and more controlled perturbation of the incumbent. For other algorithms developed for the bandwidth minimization problem, see the survey paper [3] and also [5,8].

3. Variable neighbourhood search for bandwidth minimization

The basic idea of the Variable Neighbourhood Search (VNS) metaheuristic [11,12,14,20] is a systematic change of neighbourhood within the local search algorithm. VNS proceeds by a descent method to a local minimum, then explores, systematically or at random increasingly distant neighbourhoods $N_k(x)$ of this solution. Each time it does so, one or several points within the current neighbourhood are used as initial solutions for a local descent ($x' \in N_k(x)$). One jumps from the current solution to a new one if and only if a better solution has been found. So VNS is not a trajectory following method (as Simulated Annealing and Tabu Search are) and does not specify forbidden moves. Despite its simplicity it proves to be effective. VNS exploits systematically the following observations: (i) a local minimum with respect to one neighbourhood structure is not necessarily the same with respect to another; (ii) a global minimum is a local minimum with respect to all possible neighbourhood structures; (iii) for many problems local minima with respect to one or several neighbourhoods are relatively close to each other.

The main loop of the VNS metaheuristic contains three steps: *Shaking*, *Local search* and *Neighbourhood change*. Initially the neighbourhood index k is set to k_{min} and in the shaking step a solution x' is generated within $N_k(x)$. Then a local search method is applied from x' to obtain a local optimum x'' . In the third step, if x'' is better than x , x is replaced with x'' and k is set to k_{min} ; otherwise, k is incremented by k_{step} .

```

1 function InitialSolution(A, f)
2 for i ← 1 to |V| do (mark[i] ← false) end for
3 k ← RandomInt(1, |V|); mark[k] ← true;
4 q[1] ← k; ql ← 1; f[k] ← 1; ℓ ← 1;
5 while ℓ < |V| do
6     r ← 0;
7     for i1 ← 1 to ql do
8         i ← q[i1];
9         for j1 ← 1 to |A[i]| do
10             j ← a[i, j1];
11             if not mark[j] then
12                 r ← r + 1; s[r] ← j; mark[j] ← true;
13             end if
14         end for
15     end for
16     j* ← RandomInt(1, ql);
17     for j1 ← 1 to ql do
18         j ← q[j*]; ℓ ← ℓ + 1; f[j] ← ℓ; j* ← j* + 1;
19         if j* ≥ ql - 1 then j* ← 1; end if
20     end for
21     q ← s; ql ← r;
22 end while

```

Fig. 3. Initial solution function: randomized depth-first-search.

(if $k \geq k_{\max}$, k is set to $k_{\min}$). The method repeats these three steps until a *maxiter* number of consecutive iterations is performed with no further improvement. We now present in more details the steps of a VNS based heuristic for solving MBMP.

3.1. Initialization

In this step we wish to obtain an initial solution of good quality. To this end, we construct a sequence of vertices q in depth-first-search manner and label them as follows: $1, 2, \dots, n$. The idea is based on the obvious observation that, in a good (small maximum bandwidth) solution, adjacent vertices should have close labels. In addition, our procedure should allow us to generate different solutions in different restarts. To reach this goal, we introduce randomness in the search.

Breadth-first-search vertices are placed in rows as follows: the first row contains only vertex k (selected at random) and label 1 is assigned to it ($f(k) \leftarrow 1$); vertices adjacent to k are in the second row, and their labels are $2, 3, \dots$; the third row contains vertices not included in previous rows but connected with vertices in the second row and so on. Obviously, connected vertices cannot have a bandwidth larger than their larger vertex degree, and thus, the bandwidth of our initial solution is bounded by the maximum vertex degree of G .

In addition, in order to be able to start the VNS heuristic with different starting solutions, we introduce randomness, as follows. Assume that there are n_i vertices $v_{ij}, j = 1, \dots, n_i$, at row (level) i . We take the random index j^* between 1 and n_i to assign the smallest label to the vertex v_{ij^*} . Then the next label is assigned to the vertex with index $(i, j^* + 1)$, then to $(i, j^* + 2)$, etc. The last label in row i is assigned to one with the index $(i, j^* - 1)$. A detailed pseudo-code is given in Fig. 3.

In algorithm `InitSolution` graph G is given, using the adjacency list $A = (A(1), \dots, A(n))^T$, where $A(i) = (a(i, 1), \dots, a(i, |A(i)|))$ represents vertices adjacent to vertex i . In the final labelling

$$\begin{pmatrix} q(1) & q(2) & \dots & q(n) \\ f(1) & f(2) & \dots & f(n) \end{pmatrix} = \begin{pmatrix} q(1) & q(2) & \dots & q(n) \\ 1 & 2 & \dots & n \end{pmatrix},$$

$q(i), i = 1, \dots, n$ represents sequence of vertices found by the depth-first-search algorithm from Fig. 3.

Each vertex of the graph is visited only once; thus, the complexity of our `InitSolution` routine is $O(n)$. Once vertex j (adjacent to i) is visited, it is marked ($mark[j] \leftarrow \text{true}$), the number of marked vertices (adjacent to i) is increased ($r \leftarrow r + 1$) and the sequence of such vertices s is augmented. Then labelling f is performed, as described earlier and sequence q is augmented ($q \leftarrow s$), together with its cardinality ($ql \leftarrow r$).

3.2. Shaking

To perform a shaking step within VNS, we need to define the distance between any two solutions f and f^0 . Usually the neighbourhood of f is defined by using a 1-interchange move: all possible pairs of vertex labels (one at a time) are interchanged. We say that the labelling f' belongs to the k th neighbourhood of the labelling f ($f' \in \mathcal{N}_k(f)$) if f and f' differ in $k + 1$ labels (colors). Then the distance ρ is defined as

$$\rho(f, f') = k \iff f' \in \mathcal{N}_k(f).$$

```

1  function Shake-1(k, f)
2  Let  $K = \{v \mid B(f, v) \geq B'\}$ , where  $B'$  is selected such that  $|K| \geq k$ .
3  for  $i = 1$  to  $k$  do
4       $u \leftarrow \text{RandomInt}(1, |K|)$ ;
5       $v \leftarrow \text{such that } |f(v) - f(u)| = B(f, u)$ ;
6      if  $((u, v) \in E)$  then
7           $w \leftarrow \arg \min_w \{\max\{f_{\max}(w) - f(v), f(v) - f_{\min}(w)\} \mid$ 
8               $\mid f_{\min}(u) < f(w) < f_{\max}(u)\}$ ;
9          swap  $(f(v), f(w))$ 
10     endif
11 end for

```

Fig. 4. Shake 1 function (reduced swap neighbourhood).

Note that two solutions may differ in a minimum of two labels. In this case their distance k is equal to 1 ($k + 1 = 2$). Note also that the number of 1-interchange steps for reaching f' from f should be $\leq k$. For example, the distance between solutions $f = (f(1), f(2), f(3), f(4)) = (1, 2, 3, 4)$ and $f' = (f'(1), f'(2), f'(3), f'(4)) = (3, 4, 1, 2)$ is equal to 3, since they differ in all four labels. In order to reach f' from f , we need to perform two interchanges: swap 1 with 3 and 2 with 4. Observe that the distance ρ may be seen as a Hamming distance

$$\rho(f, f') = \sum_{i=1}^n \eta(i) - 1, \eta(i) = \begin{cases} 1 & f(i) = f'(i) \\ 0 & \text{otherwise} \end{cases}$$

Our first shaking procedure is presented in Fig. 4. It uses the distance function ρ defined above. k different edges of the graph are selected at random and the labels on their end-points are exchanged.

In the first step of Shaking-1 routine, we find a set of vertices $K \subseteq V$ which will be used in swapping. In order to be sure that its cardinality is larger than k , we find the threshold value B' such that $|\{v \mid B(f, v) \geq B'\}| \geq k$. In this way we also collect vertices which have the largest bandwidth; this fact is of course desirable since we would like to reduce the bandwidth of the matrix. Vertex $u \in K$ is selected at random and its critical vertex v found (or selected at random if more than one is found): $|f(u) - f(v)| = B(f, u)$. Next we find a vertex w to swap with v such that the maximum difference of $f(u)$ and $f(v)$ is minimal: find w such that $\max\{f(v) - f_{\min}(w), f_{\max} - f(v)\}$ is minimum, where $f_{\min}(u) \leq f(w) \leq f_{\max}(u)$ and

$$f_{\max}(y) = \max\{f(z) \mid (y, z) \in E\}, \quad (2)$$

$$f_{\min}(y) = \min\{f(z) \mid (y, z) \in E\}. \quad (3)$$

Finally we swap the labels of vertices v and w .

It is expected that the shaking step will diversify the incumbent solution, i.e., it will produce a solution which moves away from the current solution f . We found that Shake-1 is not powerful enough to be used alone: sometimes it is not easy to escape from a deep local minimum. In order to improve diversification, we consider the second shaking procedure based on the rotation of two neighbourhoods [25].

Our second shaking function uses a presentation of the solution as $\pi = f^{-1}$. Note that this transformation from f to π (or from π to f) can be made as follows: $\pi(f(v)) = v, \forall v$ (or $f(\pi(v)) = v, \forall v$). We choose at random two indices i and j ($i < j$) and some k in between, i.e., we have

```

1  function Shake-2(k, f)
2  for  $i = 1$  to  $k$  do
3       $b \leftarrow \text{RandomInt}(1, n)$ ;  $e \leftarrow b + \text{RandomInt}(2, \min(20, B(f)/2))$ ;
4      if  $e > n$  then  $e \leftarrow n$  end if;
5       $m \leftarrow \text{RandomInt}(1, e - b - 1)$ ;
6      for  $j = b$  to  $e$  do
7          Let  $f_{i_j}$  be a number such that  $f[f(i_j)] = j$ ;
8          if  $j \geq b + m$  then  $f[f(i_j)] \leftarrow j - m$  else  $f[f(i_j)] \leftarrow j + e - b - m + 1$  end if;
9      end for
10 end for

```

Fig. 5. Shake 2 function (rotation).

```

1  function Shake( $f, k, k_{\min}, k_{\text{step}}$ )
2  specify  $k'_{\max}$ ;
3  if  $k \leq k'_{\max}$  then
4      Shake1( $k, f$ );
5  else
6       $k' \leftarrow \text{Int}((k - k_{\min})/k_{\text{step}})$ 
7      Shake2( $k', f$ );
8  endif

```

Fig. 6. Shake function.

$$f: \begin{pmatrix} \cdots & v_i & v_{i+1} & \cdots & v_{k-1} & v_k & v_{k+1} & \cdots & v_j & \cdots \\ \cdots & i & i+1 & \cdots & k-1 & k & k+1 & \cdots & j & \cdots \end{pmatrix}.$$

Then we rotate the labels for $i - k$ to get the following new labels f' :

$$f': \begin{pmatrix} \cdots & v_k & v_{k+1} & \cdots & v_j & v_i & v_{i+1} & \cdots & v_{k-1} & \cdots \\ \cdots & i & i+1 & \cdots & j - (k - i) & j - (k - i) + 1 & j - (k - i) + 2 & \cdots & j & \cdots \end{pmatrix}.$$

More details are given in Fig. 5.

Our Shake procedure combines Shake-1 and Shake-2 as given in Fig. 6. The balance between these two algorithms is controlled by the parameter $k'_{\max}$: if $k \leq k'_{\max}$ we use Shake-1; otherwise, we diversify our search with Shake-2. Moreover, we found in preliminary computational analysis that the step between two neighbourhoods in Shake-1 should be larger than a usual value 1 (we fix it to $k_{\text{step}} = 5$ in all runs). However, the step k' for the rotation neighbourhood needs to be smaller. It is obtained as $k' = \lfloor \frac{k - k_{\min}}{k_{\text{step}}} \rfloor$, where $\lfloor g \rfloor$ is the largest integer smaller than g , and $k_{\min}$ (a parameter) is the smallest neighbourhood that we jump in perturbing the incumbent f . In fact, the current neighbourhood k in our VNS-Band heuristic is changed to $k = k_{\min}$ to $k_{\max}$ by a step k_{step} .

3.3. Local search

Local search in our VNS-Band, is based on the IHL procedure suggested in [16]. As well as being very fast, IHL is well suited as a local search procedure within the VNS heuristic [12].

We now give some definitions. Let an edge (u, v) be *critical* if $|f(u) - f(v)| = B(f)$, where f is a given labelling (or solution) and $B(f)$ its bandwidth (or value). Let us denote with $E_c(f)$ a set of critical edges for a given labelling f :

$$E_c(f) = \{(u, v) \in E : |f(u) - f(v)| = B(f)\}.$$

Vertex v of G is critical if it is the end point to at least one critical edge. We denote the critical vertex set by $V_c(f)$. Let the reduced swap neighbourhood $N(v, f)$ of the critical vertex v under f be defined as

$$N(v, f) = \{u : |mid(v) - f(u)| < |mid(v) - f(v)|, (u, v) \in E_c(f)\},$$

where

$$mid(v) = \left\lfloor \frac{f_{\max}(v) + f_{\min}(v)}{2} \right\rfloor$$

($f_{\max}(v)$ and $f_{\min}(v)$ are defined earlier in (2) and (3), respectively). We now define a reduced swap neighbourhood $N(f)$ in a natural way, i.e.,

$$N(f) = \bigcup_{v \in V_c(f)} N(v, f).$$

In this way, a set of suitable swap nodes for (u, v) , $u \in N(v, f)$, is taken to be those nodes with labels $f(u)$ which are closer to $mid(v)$ than $f(v)$. Local search with respect to $N(f)$ is performed: swapping the label of a current critical node v with the labels of nodes $u \in N(v, f) \subseteq N(f)$ is attempted one at a time in ascending values of $|mid(v) - f(u)|$ until the solution is improved (see [16] for details).

```

1  logical function Move( $f, f', \alpha$ )
2  Move  $\leftarrow$  .false.
3  if  $B(f') < B(f)$  or
4      ( $B(f') = B(f)$  and  $|V_c(f')| < |V_c(f)|$ ) or
5      ( $B(f') = B(f)$  and  $\rho(f', f) > \alpha$ ) then
6      Move  $\leftarrow$  .true.
7  end if

```

Fig. 7. Move function.

```

1  function VnsBand( $A, k_{min}, k_{max}, k_{step}, t_{max}, \alpha, f^*$ )
2   $B^* \leftarrow \infty$ ;  $t \leftarrow 0$ ;  $i_{max} = \text{Int}((k_{max} - k_{min})/k_{step})$ ;
3  while  $t < t_{max}$  do
4       $f \leftarrow \text{InitSol}(A, f)$ ;  $f \leftarrow \text{LocalSearch}(f)$ ;
5       $i \leftarrow 0$ ;  $k \leftarrow k_{min}$ ;
6      while  $i \leq i_{max}$  do;
7           $f' \leftarrow \text{Shake}(f, k, k_{min}, k_{step})$ ;
8           $f' \leftarrow \text{LocalSearch}(f')$ ;
9          if  $\text{Move}(f, f', \alpha) = \text{.true.}$  then
10              $f \leftarrow f'$ ;  $k \leftarrow k_{min}$ ;  $i \leftarrow 0$ ;
11         else
12              $k \leftarrow k + k_{step}$ ;  $i \leftarrow i + 1$ ;
13         end if
14     end while
15     if  $B(f) < B^*$  then  $B^* \leftarrow B(f)$ ;  $f^* \leftarrow f$ ; end if;
16      $t \leftarrow \text{CpuTime}()$ 
17 end while

```

Fig. 8. VNS-Band heuristic.

3.4. Neighbourhood change

Neighbourhood change is a very important VNS step. Several different neighbourhood change (or “Move or not”) strategies within VNS have been suggested in the literature (e.g., see [13] for the recent survey of VNS methods). To perform this move, we need to answer two questions: (i) whether to move or not: what criteria to use in changing the incumbent solution; and (ii) next neighbourhood: what will be the next neighbourhood in a case when the solution is changed (move) and in a case when the incumbent is not changed.

(i) *Move or not.* In solving MBMP we use three acceptance criteria for moving from f to f' labelling ($f \leftarrow f'$): (1) the simplest one is based on the objective function values: if ($B(f') < B(f)$); and (2) if the objective function values of f and f' are the same ($B(f') = B(f)$), but the total number of critical vertices of f' is smaller ($|V_c(f')| < |V_c(f)|$); (3) if ($B(f') = B(f)$) and $|V_c(f')| = |V_c(f)|$, but f and f' are relatively far from each other and $\rho(f, f') > \alpha$, where α is an additional parameter (we fix it to $\alpha = 10$ in all runs below). The idea was suggested earlier of a move which accepts a worse solution than the incumbent, if it is very far from it, (i.e., move if $B(f') - \alpha \cdot \rho(f, f') < B(f)$); this was called Skewed VNS [12]. However, here we move to a solution with the same value only if its Hamming distance from the incumbent is greater than α .

Table 2
“CPU2000” results published by standards performance evaluation corporation.

Processor	INTEL P.IV 3.0 GHz	INTEL P.IV 2.8 GHz	INTEL P.IV 1.8 GHz	INTEL P.IV 1.6 GHz	AMD k7-Athlon 1.2 GHz
CPU speed	1088	976	633	584	409
Factor	1088/633 = 1.72	976/633 = 1.54	633/633 = 1	584/633 = 0.92	409/633 = 0.65
Method	TS, GRASP-PR, SS-TS, PSO-HC	SA	VNS	GA, NA_a , NA_b	Summary figure

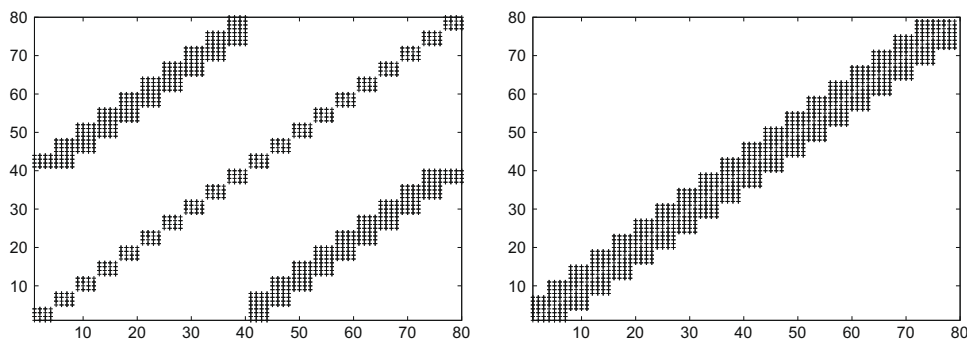


Fig. 9. Instance stream3 before and after bandwidth reduction.

Table 3

Average results of three VNS versions on the Harwell–Boeing instances.

	VNS-3			VNS-2			VNS		
	Best	Average	Worst	Best	Average	Worst	Best	Average	Worst
33 instances	22.58	29.47	40.52	21.97	22.00	22.15	21.97	22.02	22.12
80 instances	94.41	105.13	122.27	93.09	94.07	95.25	92.71	93.64	95.59

Table 4

Comparison: VNS vs. multistart local search (MLS).

Instance	VNS	MLS	Instance	VNS	MLS	Instance	VNS	MLS	Instance	VNS	MLS	Instance	VNS	MLS	Instance	VNS	MLS
ash85	9	9	lms_131	20	21	494_bus	30	45	bcsstk04	37	37	lund_a	23	23	ash292	19	22
arc130	63	63	mcca	37	39	bcsstk04	24	27	bcsstk01	5	5	lund_b	23	23	bcsstk19	14	16
bcsstk02	7	7	nos1	3	3	bcsstk06	45	47	bcsstk03	10	11	nos4	10	10	bp_1000	286	332
bcsstk01	16	16	pores_l	7	8	bp_400	270	314	bcsstk05	20	20	steam3	7	7	can_292	38	43
bcsstk22	10	12	west0132	32	35	can_838	87	101	can_144	13	13	west0156	36	39	dwt_245	21	24
can_161	18	22	west0167	34	36	dwt_419	25	30	curtis54	10	10	will199	65	71	dwt_592	29	33
dwt_234	12	15	will57	6	6	dwt_878	25	33	fs_183_1	60	66	gent113	27	29	nos3	47	61
gre_115	23	25	plat362	34	37	nos5	63	66	gre_185	21	22	ibm32	11	11	shl_400	231	258
impcol_b	20	20	west0381	151	165	str_0	116	128	impcol_c	30	32	west0497	86	96			

Table 5

Global results for the Harwell–Boeing instances.

	TS	GRASP-PR	SS-TS	GA	NS _a	NS _b	PSO-HC	SA	VNS
33 instances with $n = 30, \dots, 199$									
$B_f(G)$	22.52	22.52	22.67	22.48	22.36	22.18	22.48	22.62	22.02
% L.B. Deviation	11.25	11.16	11.81	12.70	10.18	8.72	10.15	9.20	8.56
# of best	16	20	15	18	24	27	–	29	31
Pentium 4-GHz	3.0	3.0	3.0	1.6	1.6	1.6	3.0	2.8	1.8
No scaled time in seconds	4.50	2.85	22.50	2.54	2.18	16.69	10.21	11.18	8.17
Scaled time in seconds	7.74	4.90	38.70	2.33	2.00	15.35	17.55	17.27	8.17
80 instances with $n = 200, \dots, 1000$									
$B_f(G)$	96.16	99.09	96.01	97.01	97.61	95.60	99.27	95.70	93.64
% L.B. Deviation	28.86	34.54	26.16	34.24	29.37	25.43	28.44	27.02	22.72
# of best	12	12	24	10	24	30	–	28	75
Pentium 4-GHz	3.0	3.0	3.0	1.6	1.6	1.6	3.0	2.8	1.8
No scaled time in seconds	139.42	151.17	456.27	85.22	240.64	750.97	186.40	199.25	136.71
Scaled time in seconds	239.80	260.07	794.43	78.40	222.53	690.89	320.68	290.79	136.71

Next neighbourhood. We introduce parameters in order to better control the neighbourhood change during the search for a better solution: k_{min} – the closest to the incumbent neighbourhood for shaking ($\rho(f, f') = k_{min}$); k_{step} – the difference between two successive neighbourhoods for shaking; k_{max} – the furthest neighbourhood for shaking. If a move is made, we continue the search in k_{min} ; otherwise, the next neighbourhood is equal to the current one plus k_{step} .

3.5. VNS-Band

As noted earlier, we found that applying the basic VNS (with a single parameter k_{max} and Hamming distance) would not produce an effective and efficient heuristic for solving all MBMP instances. Therefore we slightly extended the basic VNS by adding two new parameters k_{min} and k_{step} . They easily control the change of neighbourhoods (induced from the Hamming distance) during the search. Such extended VNS has already been successfully applied in finding a maximum clique of the graph [15] and k -cardinality tree problems on the graph [26,1] (see Fig. 7).

In addition, for the first time, we suggest a multi-start VNS. With preliminary testing we found that, in solving some hard MBMP instances, it is better to try with a new initial solution than to perturb the incumbent (using both shaking procedures). To put it simply, we found that only in this way were we sometimes able to successfully proceed with the search. This may be explained by the fact that MBMP local minima are very far from each other in such cases (this situation is not usual for many other combinatorial optimization prob-

Table 6

Average results of three methods on the Harwell–Boeing instances with 33 and 80 instances.

	PSO-HC			SA			VNS			B(VNS)–B(SA)		
	Best	Average	Worst	Best	Average	Worst	Best	Average	Worst	Best	Average	Worst
33 Instances	22.39	22.48	22.55	22.06	22.62	23.36	21.97	22.02	22.12	–0.09	–0.60	–1.24
80 Instances	98.80	99.27	99.66	94.80	95.70	97.44	92.71	93.64	95.59	–2.09	–2.06	–1.85

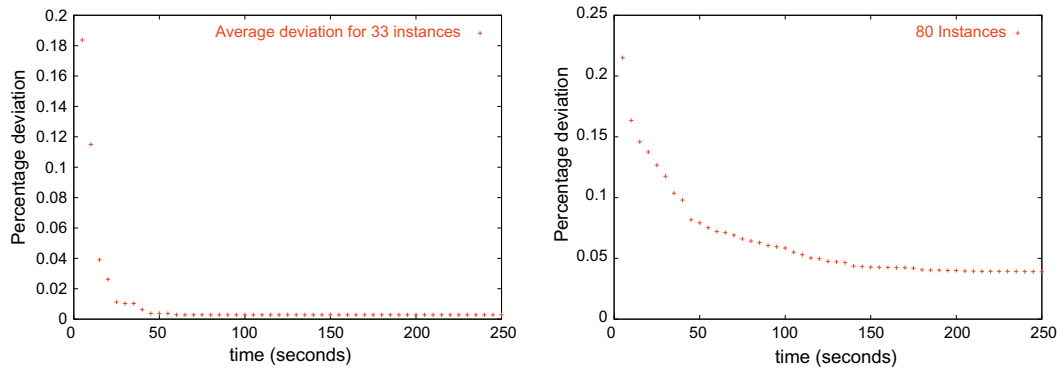


Fig. 10. Thirty-three and 80 instances.

lems). Thus, we introduce into VNS a restart option as a diversification tool. A detailed pseudo-code is given in Fig. 8, where the restart is performed in Step 4. It uses randomized breath first search procedure `InitSol()` followed by the local search `LocalSearch()`. In Computational result section the VNS variant without step 4 will be denoted as VNS-3. We note that the simple yet effective combination of diversification approaches we use may be similarly employed to enhance the performance of other approaches.

4. Computational results

Methods compared. We compare the solution qualities (minimum bandwidth obtained) and running times of our VNS based heuristic with six other approaches from the literature: TS (Tabu Search) [19], GRASP-PR (Greedy Randomized Adaptive Search Procedure–Path Relinking) [24], SS-TS (Scatter Search–Tabu Search) [2], GA (Genetic Algorithm) [16], NS_a , NS_b (Node Shift) [16], and SA (Simulated Annealing) [25]. Based on previous results reported, the SA heuristic may be seen as the state-of-the-art heuristic. In this section we will show that our VNS-Band may be considered the new state-of-the-art heuristic for MBMP.

Computer support. VNS algorithms were implemented in C and compiled with Microsoft Visual C++ 6.0 and the program was run into a Pentium 4 at 1.8 GHz with 1 GB of RAM. Different methods compared in this section were coded at different computers. In order to deal

Table 7
Detailed results for the 33 medium instances from the Harwell–Boeing Sparse Matrix Collection.

Instance	n	LB	Best	TS		GRASP-PR		SS-TS		GA		NS_b		SA		VNS		VNS vs. All Δ
				B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	
arc130	130	63	63	64	12.09	63	0.97	63	57.79	63	2.38	63	15.94	63	23.20	63	0.02	0
ash85	85	9	9	9	0.77	9	0.28	10	2.97	9	1.00	9	3.72	9	1.10	9	3.35	0
bcsprw01	39	5	5	5	0.20	5	0.07	5	0.61	5	0.45	5	0.48	5	0.40	5	0.42	0
bcsprw02	49	7	7	7	0.32	7	0.41	7	1.39	7	0.64	7	7.00	7	0.20	7	0.24	0
bcsprw03	118	9	10	11	1.08	11	0.72	11	5.40	12	1.17	11	5.74	10	1.20	10	1.44	0
bcsstk01	48	16	16	17	0.93	16	0.74	17	2.76	16	1.17	16	2.42	16	0.60	16	0.29	0
bcsstk04	132	36	37	38	12.26	37	3.29	38	55.05	37	4.19	37	31.58	37	79.20	37	0.04	0
bcsstk05	153	19	20	20	5.46	20	4.84	21	25.28	21	2.13	20	20.48	20	10.60	20	0.20	0
bcsstk22	110	9	10	11	0.81	10	1.18	11	2.81	11	1.66	10	4.77	11	5.50	10	37.37	0
can_144	144	13	13	14	1.56	14	2.15	14	7.80	15	3.20	13	10.97	13	17.60	13	0.23	0
can_161	161	18	18	19	1.97	18	0.49	18	13.80	19	2.14	18	22.63	18	3.00	18	0.48	0
curtis54	54	10	10	10	0.58	10	0.48	10	2.77	10	0.84	10	1.49	10	0.50	10	0.02	0
dwt_234	117	11	11	11	1.09	11	1.50	11	6.34	12	1.38	11	5.95	11	1.10	12	10.26	1
fs_183_1	183	52	60	61	24.91	61	8.00	63	127.82	61	4.97	62	37.63	61	8.70	60	14.25	–1
gent113	104	25	27	27	3.36	27	0.63	28	11.51	27	1.63	27	11.42	27	3.90	27	2.13	0
gre_115	115	20	23	24	2.46	24	2.44	24	9.42	23	2.25	24	12.41	23	1.60	23	15.94	0
gre_185	185	17	21	22	2.82	22	4.51	22	20.98	22	2.83	21	40.78	22	6.80	21	1.14	0
ibm32	32	11	11	12	0.31	11	0.22	11	1.49	11	0.88	11	0.97	11	0.30	11	0.03	0
impcol_b	59	19	20	21	1.77	21	0.75	21	4.35	20	1.23	20	3.36	20	1.20	20	0.14	0
impcol_c	137	23	30	31	4.52	31	3.33	32	19.89	31	4.89	31	19.91	30	3.10	30	9.81	0
lms_131	123	18	20	21	2.73	22	1.91	21	14.85	20	2.55	20	11.20	20	1.80	20	2.54	0
lund_a	147	19	23	23	3.90	23	3.57	23	30.39	23	2.88	23	21.36	23	40.60	23	0.02	0
lund_b	147	19	23	23	4.06	23	3.57	23	22.60	23	3.19	23	21.36	23	40.80	23	0.01	0
mcca	168	32	37	37	13.47	37	6.60	37	85.73	37	4.22	37	40.34	37	81.80	37	30.21	0
nos1	158	3	3	3	0.88	3	1.96	3	7.57	4	1.13	3	6.44	3	1.10	3	0.00	0
nos4	100	10	10	10	0.98	10	1.13	10	3.36	10	1.03	10	5.50	10	0.90	10	0.89	0
pores_1	30	7	7	7	0.24	7	0.21	7	0.93	7	0.50	7	0.63	7	3.10	7	0.89	0
steam3	80	7	7	7	1.27	7	0.72	7	4.33	7	1.39	7	4.44	7	8.90	7	0.00	0
west0132	132	23	32	34	7.20	35	6.30	34	25.63	33	3.66	34	18.84	33	5.40	32	42.71	–1
west0156	156	33	36	37	6.71	37	6.52	38	32.52	37	4.58	37	33.16	36	2.80	36	12.73	0
west0167	167	31	34	34	5.74	35	4.28	35	30.68	36	2.55	34	26.63	34	4.80	34	69.28	0
will199	199	55	65	67	21.53	69	20.11	67	102.24	66	14.31	66	106.20	65	6.10	65	11.28	0
will57	57	6	6	6	0.48	7	0.30	6	1.46	7	0.89	1	0.22	6	1.10	6	1.25	0
Average		19.84	21.94	22.52	4.50	22.52	2.85	22.67	22.50	22.48	2.54	22.18	16.69	22.06	11.18	21.97	8.17	–0.03

with this problem, we use scale factors which compare the performances of the computers used. Thus, we report CPU times in the tables below as if all methods were run on the same computer. Running times reported in the papers mentioned are multiplied by the corresponding factor in Table 2 (see CPU speed benchmarks available at <http://www.spec.org/cpu2000/>).

Test instances. Following [2,16,17,19,24,25] we compiled 113 instances from the Harwell–Boeing Sparse Matrix Collection (<http://math.nist.gov/MatrixMarket/data/Harwell-Boeing>). They are divided into two sets; the first set consists of 33 instances with the number of nodes ranging from 30 to 199 and the second set has 80 instances with the number of nodes ranging from 200 to 1000. All instances are matrices arising from linear systems, least squares, eigenvalue calculations and other real scientific and engineering problems. In Fig. 9 one such matrix (and its equivalent form after bandwidth reduction) is illustrated.

Preliminary testing. There are several critical issues in applying our VNS based heuristic for solving MBMP and we address some of them below.

- (i) *VNS parameters.* As mentioned earlier, for the VNS, we always use the following parameter values: $k_{min} = k_{step} = 5$, $k'_{max} = 100$, and $k_{max} = 200$. The stopping condition is chosen to be $t_{max} = 500$ seconds. Those values are obtained after a brief computational analysis.
- (ii) *VNS variants.* Beside the VNS based heuristic given in Fig. 8, we tested another two variants: (a) VNS without $Shake\ 2()$ (VNS-2); (b) VNS without restart (VNS-3). We found experimentally that the natural shaking procedure (based on the Hamming distance, $Shake\ 1()$), was not sufficient for obtaining solutions of good quality for some large problems. They are for example almost all bp instances: bp_1000 , bp_1200 , etc. However, for some problem instances, better results were obtained with VNS-2. Moreover, for instances 685_bus, bp_0 , nos_3 and str_0 the best known new results are obtained with VNS-2 (the new best values are 32, 236, 45 and 116 for these instances, respectively). In Table 3 summary results on all 113 instances with VNS and VNS-2 are reported. It is clear that there is no significant difference between them. In other words, the perturbation of the incumbent based on the rotation step is not very important in our VNS based heuristic. Interesting results are obtained with VNS-3, where line 4 is simply removed from Algorithm 8. Note that the basic VNS heuristic does not usually contain a restart step (presented at line 4), i.e., the basic VNS is in fact VNS-3. The best

Table 8

Eighty large instances from the Harwell–Boeing sparse matrix collection (Part I).

Instance	n	LB	Best	TS		GRASP-PR		SS-TS		GA		NS _b		SA		VNS		VNS vs. All
				B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	
494_bus	494	25	29	31	21.72	35	10.71	31	78.40	47	14.70	32	256.83	32	24.40	29	20.08	–2
662_bus	662	36	39	40	49.51	44	25.99	41	189.34	47	44.73	43	363.55	43	70.40	39	71.40	–1
685_bus	685	30	32	35	31.94	46	9.62	34	439.83	42	50.19	40	304.68	36	62.00	34	57.53	0
ash292	292	16	19	20	5.96	22	6.92	20	77.01	21	3.92	20	139.18	21	34.40	19	39.35	–1
bcsppwr04	274	23	24	25	8.20	26	3.72	25	69.49	29	4.55	26	134.89	25	28.00	24	33.52	–1
bcsppwr05	443	25	27	29	16.66	35	12.89	28	121.21	41	12.23	31	202.84	30	26.80	27	28.55	–1
bcsstk06	420	38	45	48	26.78	50	33.85	46	229.07	52	9.75	46	360.37	45	247.90	45	208.90	0
bcsstk19	817	13	14	16	26.97	16	66.74	16	733.97	17	54.81	15	312.20	15	222.90	14	199.34	–1
bcsstk20	467	8	13	18	6.40	19	8.36	14	132.99	14	4.34	13	150.88	14	44.60	13	52.13	0
bcsstm07	420	37	45	46	23.23	48	76.93	46	286.49	50	14.55	46	365.44	48	215.70	45	209.52	–1
bp_0	822	174	236	245	375.65	258	378.05	245	949.62	239	168.94	244	1169.75	240	140.70	237	123.61	–2
bp_1000	822	197	287	302	550.50	297	629.91	303	938.09	290	256.59	298	1746.79	291	216.20	287	231.70	–3
bp_1200	822	197	291	303	552.61	303	629.82	299	932.50	295	218.48	301	1693.90	296	218.20	291	179.18	–4
bp_1400	822	199	293	311	519.79	313	513.10	309	968.95	297	258.58	305	1739.60	300	217.40	293	214.28	–4
bp_1600	822	199	294	310	524.10	317	539.68	312	991.20	300	256.64	306	1751.63	299	231.80	294	242.63	–5
bp_200	822	186	258	269	439.06	271	413.21	272	946.98	268	204.59	268	1481.67	267	168.40	258	179.27	–9
bp_400	822	188	269	281	426.45	285	406.24	280	955.66	276	190.34	277	1648.59	276	180.00	269	204.65	–7
bp_600	822	190	274	290	485.66	297	399.88	293	929.34	280	266.74	285	1627.16	279	193.30	274	158.82	–5
bp_800	822	197	283	294	489.20	307	454.38	301	960.45	292	279.38	292	1721.92	286	219.90	283	247.75	–3
can_292	292	34	38	40	13.38	42	5.55	40	99.05	43	8.39	39	183.72	41	61.50	38	55.40	–2
can_445	445	46	52	57	26.72	58	36.86	54	284.47	61	11.48	54	385.30	56	114.00	52	119.68	–2
can_715	715	54	72	76	58.54	78	9.79	72	752.31	74	60.66	74	423.58	74	223.00	72	192.68	0
can_838	838	75	86	89	104.19	88	24.59	89	918.15	92	96.20	91	498.23	88	384.20	86	402.23	–2
dwt_209	209	21	23	24	4.82	24	0.84	24	39.63	25	2.95	23	126.88	23	29.20	23	25.30	0
dwt_221	221	12	13	14	3.09	13	5.31	13	19.70	15	2.25	13	99.20	13	23.40	13	23.88	0
dwt_245	245	21	21	23	6.28	26	11.10	22	49.13	24	3.34	22	124.51	24	9.30	21	10.20	–1
dwt_310	310	11	12	12	3.07	12	6.47	12	97.78	12	4.25	12	148.25	12	13.00	12	11.45	0
dwt_361	361	14	14	15	5.13	15	0.58	14	41.33	18	4.73	14	218.60	14	8.30	14	7.22	0
dwt_419	419	23	25	27	11.20	29	21.28	27	106.22	30	7.58	26	220.69	26	59.90	25	69.21	–1
dwt_503	503	29	41	43	28.41	45	5.32	43	218.82	43	36.69	42	433.64	44	163.50	41	174.40	–1
dwt_592	592	22	29	32	22.53	33	85.53	31	274.51	32	41.19	30	341.31	32	123.70	29	111.32	–1
dwt_878	878	23	25	26	41.70	35	77.83	26	300.77	36	84.39	27	622.09	26	106.70	25	104.70	–1
dwt_918	918	27	32	33	35.78	36	9.04	34	463.90	35	93.30	34	548.16	33	243.00	32	223.19	–1
dwt_992	992	35	35	47	63.49	49	206.14	40	909.36	46	98.53	40	960.21	35	116.40	37	124.59	2
fs_541_1	541	270	270	270	25.33	270	16.20	270	45.88	270	23.70	270	303.90	270	82.40	270	93.02	0
fs_680_1	680	17	17	17	20.62	17	28.06	17	171.66	17	40.89	17	337.90	17	39.80	17	42.71	0
fs_760_1	760	36	38	39	60.39	39	76.32	39	914.72	38	63.95	38	584.08	38	95.60	38	79.35	0
gr_30_30	900	31	33	37	50.84	58	71.85	35	598.26	50	116.00	43	746.44	45	370.10	33	443.13	–2
gre_216a	216	17	21	22	2.43	21	6.95	21	23.75	21	3.41	21	146.82	21	3.50	21	3.77	0
gre_343	343	23	28	29	4.72	29	15.90	28	82.90	29	5.47	28	256.99	28	7.40	28	8.09	0
gre_512	512	30	36	37	13.18	36	71.23	36	191.27	36	45.17	36	434.79	36	14.70	36	13.54	0
hor_131	434	46	55	57	25.13	64	20.67	56	182.61	61	14.33	56	331.69	55	154.10	55	167.39	0
impcol_a	206	30	32	33	6.94	34	2.29	33	28.00	33	3.05	32	136.11	32	5.40	32	6.24	0
impcol_d	425	36	40	40	21.15	42	24.11	40	97.37	48	6.02	40	269.02	42	77.50	40	92.37	0
impcol_e	225	34	42	43	11.87	42	2.51	43	60.97	42	5.17	42	168.18	42	49.60	42	43.66	0

among 20 solutions appears to be competitive with other methods, but the difference between the best and the worst solutions obtained in 20 runs is usually much greater than for other heuristics (see Table 3). Detailed results for all three VNS versions may be found on our web site: <http://www.mi.sanu.ac.yu/nenad/band/index.htm>. (On the same site one can find the executive version of our code, as well as the link to all test instances.) Thus, we conclude that combining restart and shaking will improve the diversification function of our VNS heuristic.

(iii) *Multistart LS*. If we remove lines 5–14 from VNS pseudo code given in Fig. 8, we get Multistart local search (MLS). The question is how our VNS compares with MLS, or whether we need a shaking step at all. Bandwidth values of these two heuristics obtained in one run are reported in Table 4. It appears that MLS never gives better bandwidth value than VNS. For 33 small instances, MLS reaches equal values 17 times and worse values 16 times. However, for 20 larger instances randomly selected from the set of 80, the results obtained by VNS are always better. This has to be expected since the MLS suffers from the so-called *central limit catastrophe* property, already observed in solving many other combinatorial optimization problems.

Summary tables. In Table 5, we summarize the results of all methods on all 113 test instances (made up of 33 small instances and 80 large instances). Results for TS, GRASP-PR, SS-TS, GA, NS and SA are taken from [2] and for PSO-HC from [17]. Detailed results may be found in Appendix 2. For each method the following characteristics are given: (i) the average bandwidth value $b_f(G)$; (ii) the average deviation from the lower bound (L.B. Deviation taken from [18]); (iii) the number of solutions equal to the best-known; (iv) the computer used in testing; (v) the average running times in seconds as originally reported and (vi) the average running times after scaling (as all methods were run on the same computer Pentium 4, Intel, 1.8 MzH).

From the summary Table 5 it is clear that our VNS-based heuristic outperforms all other approaches for solving both medium and large size instances: (i) its average bandwidth value $B_f(G)$ and its average % deviation above the lower bound are the smallest; (ii) the number of best known solutions is the highest; (iii) the average running time in reaching these solutions (within 500 seconds for each run) is the second smallest.

It should be noted that the results in Table 5 of all methods, except three, are obtained in one run on each instance. However, for PSO-HC, SA and VNS we report average values obtained in 5 independent runs for PSO-HC and 20 runs for SA and VNS. In Table 6 we give the average objective function values of the best, the average and the worst solutions obtained by these three methods (see Table 10 in Appendix 3 for the detailed results). The last three columns report on the difference between B(VNS) and B(SA). The negative difference indicates the better performance of our VNS based heuristic over the current state-of-the-art heuristic. Therefore we may consider a VNS based method as a new state-of-the-art heuristic for solving MBMP.

Table 9
Eighty large instances from the Harwell–Boeing sparse matrix collection (Part II).

Instance	n	LB	Best	TS		GRASP-PR		SS-TS		GA		NS _b		SA		VNS		VNS vs. All Δ
				B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	B_f	Time	
jgmesh1	936	24	27	28	14.43	27	84.95	27	347.02	28	85.52	27	528.60	27	33.80	28	30.01	1
jpwh_991	983	82	90	90	79.64	96	51.18	90	915.91	99	100.84	93	1018.64	94	104.30	90	114.39	0
lms_511	503	33	44	45	26.71	49	44.10	46	138.60	48	40.50	44	391.29	44	123.80	44	125.53	0
mbeacxc	487	248	261	263	1125.41	265	1410.48	277	1122.88	263	461.09	262	5575.55	262	1774.00	261	982.34	–1
mbeafw	487	246	261	263	1137.42	265	1411.59	277	1093.09	262	544.97	262	5607.62	261	1744.90	261	963.24	0
mbeause	492	249	254	257	901.40	256	1597.90	271	1068.66	255	460.67	255	5031.29	255	1289.50	254	892.12	0
mcfe	731	112	126	127	282.03	130	63.45	128	938.66	127	118.94	127	1390.05	126	1868.90	126	792.13	0
nnc261	261	22	24	24	5.93	25	16.91	25	27.33	28	3.17	24	117.59	25	8.70	24	7.18	0
nnc666	666	33	40	42	52.92	45	41.70	41	714.03	46	57.38	43	326.98	42	108.50	40	106.46	–1
nos2	638	3	3	3	3.61	3	12.59	3	133.39	4	25.55	3	126.60	3	114.50	3	96.98	0
nos3	960	43	45	72	76.98	79	141.56	48	934.29	63	121.59	57	971.53	62	811.60	47	660.70	–1
nos5	468	53	63	65	27.52	69	84.77	65	261.36	67	12.92	64	462.91	64	121.90	63	101.20	–1
nos6	675	15	16	17	5.40	16	33.35	16	134.84	17	36.08	16	316.32	16	12.50	16	14.62	0
nos7	729	43	65	66	19.73	66	69.05	65	256.48	67	92.34	65	736.23	65	23.90	65	28.13	0
orsirr_2	886	62	86	87	62.10	91	32.07	87	769.55	96	91.16	90	1184.36	88	164.50	86	164.94	–1
plat362	362	29	34	35	17.31	36	2.03	36	99.17	39	10.11	34	348.78	36	189.40	34	179.58	0
plskz362	362	15	18	18	6.08	20	5.51	19	46.96	21	5.77	18	170.54	19	20.90	18	22.29	0
pores_3	456	13	13	13	5.04	13	2.27	13	51.92	15	7.33	13	203.99	13	13.00	13	14.03	0
saylr1	238	12	14	16	1.55	15	6.33	14	11.84	15	3.69	14	103.43	14	2.30	14	2.23	0
saylr3	681	35	47	52	22.90	52	60.61	47	239.58	51	62.89	49	497.24	53	15.60	48	15.77	1
sherman1	681	35	47	52	22.85	52	60.64	47	239.45	55	74.27	49	502.90	52	15.60	47	17.10	0
sherman4	546	21	27	27	8.29	27	2.96	27	56.16	27	38.05	27	356.85	27	11.50	27	12.93	0
shl_0	663	211	226	232	252.15	241	70.51	234	926.52	232	46.56	240	328.29	229	211.50	226	195.65	–3
shl_200	663	220	233	238	264.16	247	59.99	239	918.95	243	49.25	247	326.26	235	213.50	233	222.30	–2
shl_400	663	213	230	235	286.56	242	76.11	238	923.74	240	50.92	241	340.21	235	221.40	230	179.95	–5
steam1	240	32	44	44	9.77	46	8.48	44	121.98	44	10.09	44	239.70	44	79.10	44	90.42	0
steam2	600	54	63	65	30.05	65	113.16	63	563.07	65	142.13	63	652.74	65	687.40	63	639.17	0
str_0	363	87	116	120	90.67	124	83.09	118	832.47	120	65.91	119	484.28	119	39.90	117	43.36	–1
str_200	363	90	125	128	107.30	135	71.99	128	608.12	126	227.89	129	580.40	128	47.30	125	38.27	–1
str_600	363	101	132	135	110.45	144	57.56	135	654.86	132	104.88	135	617.64	132	55.90	132	66.59	0
west0381	381	119	151	156	142.45	159	137.78	156	911.74	153	120.45	158	246.47	153	38.10	151	35.00	–2
west0479	479	84	121	122	97.10	127	131.04	124	724.50	124	51.55	123	564.64	123	40.50	121	38.50	–1
west0497	497	69	85	87	69.74	92	71.70	87	388.91	87	19.25	87	327.57	87	137.90	85	112.57	–2
west0655	655	109	161	161	177.27	167	188.51	162	930.93	162	124.05	161	1056.66	161	80.50	163	75.85	2
west0989	989	123	210	213	359.57	217	325.17	210	940.41	212	152.58	215	1539.68	215	172.10	211	153.77	1
Average		74.19	92.62	96.16	139.42	99.09	151.17	96.01	456.27	97.01	85.22	95.60	750.97	94.80	210.26	92.71	157.33	–0.98

Table 10

VNS vs. SA on 113 Harwell–Boeing instances: best, worst and average results out of 20 restarts.

Instance	SA			VNS			Δ	Instance	SA			VNS			Δ	Instance	SA			VNS			Δ
	bst	wrt	avg	bst	wrt	avg			bst	wrt	avg	bst	wrt	avg			bst	wrt	avg	bst	wrt	avg	
arc130	63	64	63.8	63	63	63.0	0	ash85	9	9	9	9	9	9.00	0	bcspwr01	5	6	5.8	5	5	5.0	0
bcspwr02	7	8	7.8	7	7	7.0	0	bcspwr03	10	11	10.6	10	10	10.0	0	bcsstk01	16	20	18.6	16	16	16.0	0
bcsstk04	37	42	37.6	37	37	37.0	0	bcsstk05	20	22	20.8	20	20	20.0	0	bcsstk22	11	11	11.0	10	10	10.0	–1
can_144	13	13	13.0	13	13	13.0	0	can_161	18	19	18.8	18	18	18.0	0	curtis54	10	10	10.0	10	10	10.0	0
dwt_234	11	13	12.0	12	13	12.2	1	fs_183_1	61	66	62.6	60	61	60.4	–1	gent113	27	27	27	27	27	27.0	0
gre_115	23	24	23.6	23	23	23.0	0	gre_185	22	24	22.2	21	21	21.0	–1	ibm32	11	13	11.8	11	11	11.0	0
impcol_b	20	23	21.0	20	20	20.0	0	impcol_c	30	31	30.6	30	30	30.0	0	lms_131	20	21	20.6	20	20	20.0	0
lund_a	23	23	23.0	23	23	23.0	0	lund_b	23	23	23.0	23	23	23.0	0	mcca	37	38	37.4	37	37	37.0	0
nos1	3	3	3.0	3	3	3.0	0	nos4	10	10	10.0	10	10	10.0	0	pores_1	7	9	8.0	7	7	7.0	0
steam3	7	7	7.0	7	7	7.0	0	west0132	33	36	34.0	32	33	32.7	–1	west0156	36	37	36.8	36	36	36.0	0
west0167	34	35	34.4	34	35	34.2	0	will199	65	67	65.7	65	66	65.0	0	will57	6	6	6.0	6	6	6.0	0
																Avg.	22.1	23.4	22.6	22.0	22.1	22.0	–0.1
494_bus	32	37	33.8	29	31	30.0	–3	662_bus	43	44	43.2	39	41	39.5	–4	685_bus	36	38	37.0	34	36	35.0	–2
ash292	21	21	21.0	19	19	19.0	–2	bcspwr04	25	25	25.0	24	24	24.0	–1	bcspwr05	30	32	31.4	27	28	27.8	–3
bcsstk06	45	45	45.0	45	45	45.0	0	bcsstk19	15	16	15.2	14	14	14.0	–1	bcsstk20	14	18	14.9	13	13	13.0	–1
bcsstm07	48	50	48.4	45	45	45.0	–3	bp_0	240	251	241.7	237	242	239.6	–3	bp_1000	291	295	292.2	287	297	289.6	–4
bp_1200	296	299	297.2	291	297	293.8	–5	bp_1400	300	306	302.8	293	300	296.7	–7	bp_1600	299	308	300.6	294	306	299.3	–5
bp_200	267	272	268.7	258	269	263.8	–9	bp_400	276	282	277.8	269	276	272.4	–7	bp_600	279	287	281.8	274	284	279.8	–5
bp_800	286	289	286.9	283	291	286.6	–3	can_292	41	46	42.5	38	39	38.3	–3	can_445	56	63	57.8	52	56	54.0	–4
can_715	74	78	75.1	72	77	74.8	–2	can_838	88	89	88.2	86	88	87.0	–2	dwt_209	23	27	23.8	23	23	23.0	0
dwt_221	13	13	13.0	13	13	13.0	0	dwt_245	24	25	24.2	21	22	21.1	–3	dwt_310	12	13	12.2	12	12	12.0	0
dwt_361	14	14	14.0	14	15	14.3	0	dwt_419	26	26	26.0	25	26	25.7	–1	dwt_503	44	47	44.7	41	43	41.8	–3
dwt_592	32	33	32.4	29	30	29.0	–3	dwt_878	26	26	26.0	25	29	27.3	–1	dwt_918	33	36	34.4	32	35	33.8	–1
dwt_992	35	36	35.6	37	49	41.8	2	fs_541_1	270	270	270.0	270	270	270.0	0	fs_680_1	17	19	18.4	17	17	17.0	0
fs_760_1	38	38	38.0	38	38	38.0	0	gr_30_30	45	49	46.6	33	37	34.9	–12	gre_216a	21	21	21.0	21	21	21.0	0
gre_343	28	28	28.0	28	28	28.0	0	gre_512	36	36	36.0	36	36	36.0	0	hor_131	55	56	55.4	55	55	55.0	0
impcol_a	32	33	32.6	32	32	32.0	0	impcol_d	42	49	43.6	40	41	40.6	–2	impcol_e	42	43	42.2	42	42	42.0	0
jagmesh1	27	27	27.0	28	32	30.6	1	jpwht_991	94	103	96.6	90	96	92.0	–4	lms_511	44	46	44.8	44	45	44.6	0
mbeacxc	262	264	263.0	261	261	261.0	–1	mbea_w	261	263	261.8	261	262	261.1	0	mbeause	255	260	256.6	254	255	254.4	–1
mcfe	126	127	126.2	126	127	126.0	0	nnc261	25	25	25.0	24	24	24.0	–1	nnc666	42	43	42.2	40	42	41.0	–2
nos2	3	5	3.8	3	3	3.0	0	nos3	62	64	63.0	47	53	49.0	–15	nos5	64	64	64.0	63	64	63.1	–1
nos6	16	16	16.0	16	16	16.0	0	nos7	65	65	65.0	65	74	65.9	0	orsirr	88	89	88.4	86	93	89.2	–2
plat362	36	39	36.5	34	36	34.1	–2	plskz362	19	24	21.2	18	18	18.0	–1	pores_3	13	15	14.2	13	13	13.0	0
saylr1	14	14	14.0	14	14	14.0	0	saylr3	53	57	54.4	48	52	49.7	–5	sherman1	52	57	53.4	47	53	49.3	–5
sherman4	27	27	27.0	27	27	0.0	0	shl_0	229	231	230.2	226	230	228.8	–3	shl_200	235	241	238.4	233	238	235.9	–2
shl_400	235	239	237.0	230	235	232.8	–5	steam1	44	47	45.2	44	44	44.0	0	steam2	65	67	66.0	63	63	63.0	–2
str_0	119	121	120.0	117	119	117.6	–2	str_200	128	135	132.0	125	127	126.0	–3	str_600	132	134	132.8	132	134	132.9	0
west0381	153	157	154.4	151	154	152.0	–2	west0479	123	125	123.6	121	123	122.6	–2	west0497	87	89	88.2	85	86	85.9	–2
west0655	161	169	162.1	163	168	165.2	2	west0989	215	217	216.0	211	234	219.4	–4	Avg.	94.8	97.4	95.7	92.7	95.6	93.6	–2.1

Summary figure. The good performance of VNS is also shown in Fig. 10, where an average % deviation is presented as a function of the VNS running time. Note that this type of figure is also used in [24], where the efficiency of three methods (TS, GRASP-PR and SS-TS) was compared. The authors used AMD K7-Athlon PC computer at 1.2 GHz; therefore, in order to compare the efficiency of the two approaches, we used a scaling factor as in Table 2. It appears that in the first 40 seconds VNS is able to obtain better solutions than the other three heuristics; they need more than 165 seconds (on average) to obtain solutions of similar quality, in the 33 medium sized graphs. Similar results hold for the other 80 large instances.

5. Conclusions

We suggest a Variable Neighbourhood Search (VNS)-based heuristic to provide high quality solutions to the matrix bandwidth minimization problem (MBMP). In this regard, within the general VNS framework, we combine the following recent contributions: (i) reduced swap neighbourhood (based on the middle vertex concept) introduced in [19]; (ii) fast local search procedure (IHC) suggested in [16]; and (iii) rotation-based neighbourhood structure, proposed in [25]. In brief, we perturb the incumbent solution using both reduced swap and rotation neighbourhood structures and then apply IHC local search. In addition, we introduce *the number of critical vertices in the incumbent solution* as the second objective function. We do this because in MBMP there are many neighbourhood solutions with the same bandwidth value. An efficient *breadth-first-search* type algorithm for getting initial solutions is also given.

An extensive computational experience on a set of 113 benchmark instances from the literature shows that our VNS outperforms six other heuristics in terms of both solution quality and length of computing-time required. Moreover, we are able to improve on 42 (out of 113) of the previous best known solutions. Therefore, our VNS-Band algorithm may be considered as a new state-of-the-art heuristic.

Future work may include the use of a different type of permutation distance measures within VNS (see e.g. [27] for solving quadratic assignment problem). Different distance functions may induce several neighbourhood structures. They may be used in both the local search and the shaking. In addition, MBMP may be formulated using direct f and an inverse f^{-1} permutation of labels (local minimum with respect to one presentation of the solution f is not necessary, as with f^{-1}). Therefore, recent ideas of formulation space search used for solving circle packing problems [21,22] may also be tried out in solving MBMP.

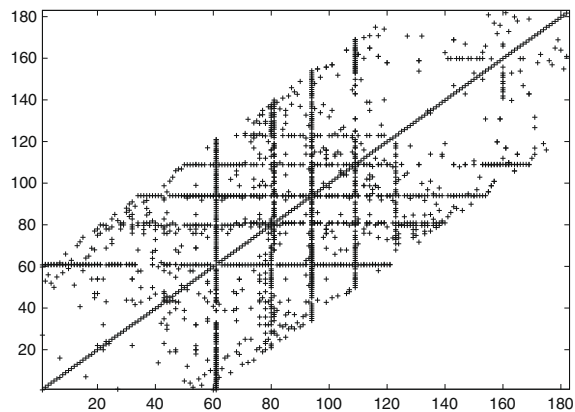
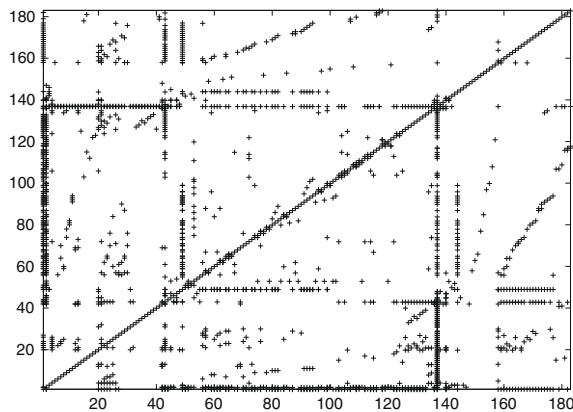
Acknowledgements

The first two authors are partially supported by the Serbian Academy of Sciences, Project number 144007. The third author is partially supported by a Spanish grant MTM2007-67433-CO2-01. We also thank Professor Pierre Hansen from GERAD and HEC Montreal for helpful suggestions which have improved the presentation of our work and Igor Grujić from the University of Belgrade for help in the computational testing.

Appendix 1. Best known solutions for two small instances

Solution for fs_183_1 ($\beta = 60$)

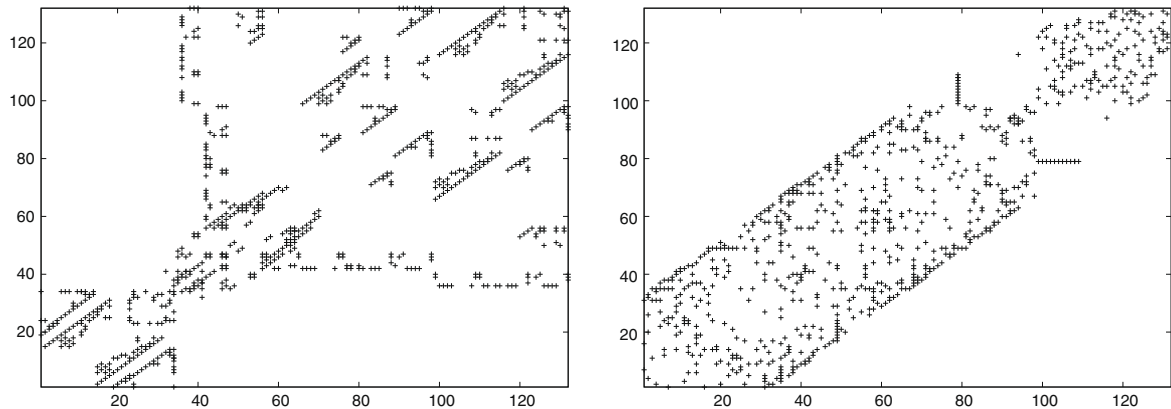
99 112 28 75 17 34 36 21 33 38 45 19 32 12 18 26 13 20 23 73 79 80 50 47 46 118 123 122 110 129 29 25 7 9 10 11 5 4 16 14 37 58 82
111 108 136 49 22 84 145 159 144 166 0 31 89 115 169 77 74 119 124 165 96 109 161 141 57 27 95 113 125 72 90 148 102 160 88 147 98
93 97 156 106 164 149 120 174 155 117 100 101 105 103 154 52 59 157 114 168 146 150 71 121 116 128 54 172 153 158 171 67 48 104 81
56 35 152 151 170 163 78 83 126 70 61 60 69 63 62 65 53 66 24 87 55 64 30 6 107 167 85 86 68 162 1 173 3 180 177 179 176 181 2 178
175 182 76 130 135 91 134 43 133 138 42 39 132 44 137 131 127 139 41 142 140 143 15 51 94 8 40 92



Solution for west0132 ($\beta = 32$)

102 110 119 129 115 106 109 101 130 100 108 107 120 104 122 123 125 127 126 113 111 117 118 105 112 128 99 131 124 121 114
116 103 79 93 49 92 70 65 69 94 35 55 63 52 62 58 95 80 72 74 97 91 90 71 56 82 85 28 67 88 96 86 87 89 66 51 54 57 98 2 53 0 16 44 59

48 50 45 36 5 15 4 25 3 1 14 32 27 46 34 8 11 7 6 13 9 31 83 20 81 26 75 33 84 30 47 29 37 73 21 18 12 22 19 10 76 17 78 64 68 61 77 60 39
42 23 24 43 41 40 38



Appendix 2. Comparison of seven methods

(See Table 7–9).

Appendix 3. Comparison of SA and VNS based heuristics

(See Table 10).

References

- [1] J. Brimberg, D. Urošević, N. Mladenović, Variable neighborhood search for the vertex weighted k -cardinality tree, *European Journal of Operational Research* 171 (2006) 74–84.
- [2] V. Campos, E. Piñana, R. Martí, Adaptive memory programming for matrix bandwidth minimization, *European Journal of Operational Research*. <<http://www.uv.es/~rmarti/paper/docs/bmp4.pdf>>.
- [3] P. Chinn, J. Chvatalova, A. Dewdney, N. Gibbs, The bandwidth problem for graphs and matrices – A survey, *Journal of Graph Theory* 6 (3) (1982) 223–254.
- [4] G.D. Corso, G. Manzini, Finding exact solutions to the bandwidth minimization problem, *Computing* 62 (3) (1999) 189–203.
- [5] G. Dueckand, J. Jeffs, A heuristic bandwidth reduction algorithm, *Journal of Combinatorial Mathematics and Computers* 18 (1995) 97–108.
- [6] T. Feo, M.G.C. Resende, Greedy randomized adaptive search procedures, *Journal of Global Optimization* 2 (1995) 1–27.
- [7] M. Garey, R. Graham, D. Johnson, D. Knuth, Complexity results for bandwidth minimization, *SIAM Journal on Applied Mathematics* (1978) 477–495.
- [8] N. Gibbs, W. Poole, P. Stockmeyer, An algorithm for reducing the bandwidth and profile of a sparse matrix, *SIAM Journal on Numerical Analysis* 13 (1976) 235–251.
- [9] F. Glover, M. Laguna, *Tabu Search*, Kluwer Academic Publishers, Boston, 1997.
- [10] E.M. Gurari, I.H. Sudborough, Improved dynamic programming algorithms for bandwidth minimization and the mincut linear arrangement problem, *Journal of Algorithms* 5 (1984) 531–546.
- [11] P. Hansen, N. Mladenović, Variable neighborhood search: principles and applications, *European Journal of Operational Research* 130 (2001) 449–467.
- [12] P. Hansen, N. Mladenović, Variable neighborhood search, in: F. Glover, G. Kochenberger, et al. (Eds.), *Handbook of Metaheuristics*, 2003, pp. 145–184.
- [13] P. Hansen, N. Mladenović, Variable neighborhood search methods, in: Floudas, Pardalos (Eds.), *Encyclopedia of Optimization*, second ed., Springer, in press.
- [14] P. Hansen, N. Mladenović, D. Pérez Brito, Variable neighborhood decomposition search, *Journal of Heuristics* 7 (2001) 335–350.
- [15] P. Hansen, N. Mladenović, D. Urošević, Variable neighborhood search for the Maximum clique problem, *Discrete Applied Mathematics* 145 (2004) 117–125.
- [16] A. Lim, B. Rodriguez, F. Xiao, Heuristics for matrix bandwidth reduction, *European Journal of Operational Research* 174 (1) (2006) 69–91.
- [17] A. Lim, J. Lin, F. Xiao, Particle swarm optimization and hill climbing for the bandwidth minimization problem, *Applied Intelligence* 26 (2007) 175–182.
- [18] R. Martí, V. Campos, E. Piñana, A branch and bound algorithm for the matrix bandwidth minimization, *European Journal of Operational Research* 186 (2008) 513–528.
- [19] R. Martí, M. Laguna, F. Glover, V. Campos, Reducing the bandwidth of a sparse matrix with tabu search, *European Journal of Operational Research* 135 (2) (2001) 211–220.
- [20] N. Mladenović, P. Hansen, Variable neighborhood search, *Computers and Operations Research* 24 (1997) 1097–1100.
- [21] N. Mladenović, F. Plastria, D. Urošević, Reformulation descent applied to circle packing problems, *Computers and Operations Research* 32 (2005) 2419–2434.
- [22] N. Mladenović, F. Plastria, D. Urošević, Formulation space search for circle packing problems, *Lecture Notes in Computer Science* 4638 (2007) 212–216.
- [23] C. Papadimitriou, The NP-Completeness of the bandwidth minimization problem, *Journal on Computing* 16 (1976) 263–270.
- [24] E. Piñana, I. Plana, V. Campos, R. Martí, GRASP and path relinking for the matrix bandwidth minimization, *European Journal of Operational Research* 153 (2004) 200–210.
- [25] E. Rodriguez-Tello, H. Jin-Kao, J. Torres-Jimenez, An improved simulated annealing algorithm for bandwidth minimization, *European Journal of Operational Research* 185 (2008) 1319–1335.
- [26] D. Urošević, J. Brimberg, N. Mladenović, Variable neighborhood decomposition search for the edge weighted k -cardinality tree problem, *Computers and Operations Research* 31 (2004) 1205–1213.
- [27] C. Zhang, Zh Lin, Zu Lin, Variable neighborhood search with permutation distance for QAP, *Lecture Notes in Computer Science* 3684 (2005) 81–88.